AeroSaga ? From-Scratch Working Guide

How the project works internally, from starting the computer to completing a simulated drone Takeoff

Project: D:\AeroSaga

Scope: Week 1 local simulation ? no real drone hardware is connected.

Read this guide in order. It explains what each service does, why it is needed, which file controls it, and what happens after you click Simulate Takeoff.

1. The project goal

AeroSaga is a drone mission-control foundation. The user selects a simulated drone in a React dashboard and requests a simulated Takeoff. The backend does not directly perform the whole operation. Instead, it starts a Temporal workflow. Temporal gives the operation a workflow ID, records its history, delivers work to a worker, and tracks the final result.

The current flow is intentionally small:

2. Three systems work together

System	Role	Where it runs
React/Vite	User dashboard, globe, button, response display	Browser + port 5173
Spring Boot	REST API, Temporal client, worker	Java process + port 8080
Temporal	Workflow history, task delivery, status	Docker container + ports 7233/8233

All three systems must be available for the Takeoff button to succeed. The frontend can display the page by itself, but the Takeoff action requires both the backend and Temporal.

3. Project structure and file responsibilities

? docker-compose.yml ? starts Temporal and defines its health check

? README.md ? basic project setup and scope

? backend/pom.xml ? Maven dependencies and Java version

? backend/src/main/resources/application.yml ? ports, Temporal address, namespace, task queue, CORS origin

? backend/.../AeroSagaApplication.java ? Spring Boot entry point

? backend/.../TemporalConfig.java ? Temporal connection, client, worker, registrations

? backend/.../MissionController.java ? REST endpoints and workflow starter

? backend/.../DroneMissionWorkflow.java ? workflow contract

? backend/.../DroneMissionWorkflowImpl.java ? workflow orchestration

? backend/.../DroneActivities.java ? activity contract

? backend/.../DroneActivitiesImpl.java ? simulated Takeoff implementation

? frontend/package.json ? npm scripts and dependencies

? frontend/vite.config.ts ? Vite and Cesium asset configuration

? frontend/src/App.tsx ? dashboard state and user interface

? frontend/src/components/Globe.tsx ? Cesium viewer and markers

? frontend/src/services/missionApi.ts ? REST request from browser to backend

? frontend/src/types.ts ? TypeScript data types

? frontend/src/styles.css ? dashboard layout and responsive styling

4. What happens when the computer starts

4.1 Docker Desktop

Docker Desktop supplies the Docker engine on Windows. The command below starts the desktop application:

Before using Compose, verify that Docker has both a Client and Server section:

If only Client appears, the Docker command-line program exists but the engine is not ready. Wait for Docker Desktop to finish starting.

4.2 Temporal container

Compose reads docker-compose.yml and starts the temporal service:

The service listens on 7233 for the backend and publishes the web UI on 8233. The container should eventually show healthy.

Temporal is the durable workflow engine. The project uses a real Temporal server, not an in-memory fake implementation.

4.3 Spring Boot backend

Maven compiles the Java project, starts embedded Tomcat, creates a Temporal connection, and starts the worker:

Successful logs include Tomcat on port 8080, WorkflowServiceStubs targeting 127.0.0.1:7233, and workflow/activity pollers using aerosaga-drone-tasks.

4.4 React/Vite frontend

npm installs browser dependencies and Vite serves the React application:

The browser opens the URL printed by Vite, normally http://127.0.0.1:5173/.

5. Backend configuration explained

application.yml contains defaults so addresses and names are not scattered through Java code:

Environment variables can override these defaults. For example:

6. Temporal concepts used by AeroSaga

Workflow

A workflow is the durable, trackable orchestration of a business operation. Here the workflow method is simulateTakeoff(String droneId). Temporal records its execution and result.

Activity

An activity performs an operation that may have external side effects. Here takeoff(String droneId) logs and returns a simulated result. Future code could replace this activity with a drone simulator or hardware adapter.

Worker

The Java worker polls the task queue and executes registered workflow and activity implementations. It is the process that performs the Java work requested by Temporal.

Task queue

aerosaga-drone-tasks is the named channel between Temporal and this worker. The workflow starter and worker must use the same name.

Namespace

default is the Temporal logical workspace used by this local project. It separates workflow records from other namespaces.

7. What happens after clicking Simulate Takeoff

Step 1 ? React reads the selected drone

App.tsx stores selectedId. When the button is clicked, the selected drone ID is passed to simulateTakeoff().

Step 2 ? Browser sends HTTP POST

missionApi.ts sends JSON to the backend:

Step 3 ? Controller creates a unique ID

MissionController builds an ID such as:

The UUID portion makes separate clicks separate workflow executions.

Step 4 ? Controller creates a workflow stub

The controller uses WorkflowClient and WorkflowOptions to target the task queue aerosaga-drone-tasks and the new workflow ID.

Step 5 ? Temporal records and schedules work

Temporal records the workflow start and places the workflow task on the task queue. The worker receives it.

Step 6 ? Workflow calls the activity

DroneMissionWorkflowImpl uses Workflow.newActivityStub with a 30-second start-to-close timeout and calls activities.takeoff(droneId).

Step 7 ? Activity returns result

DroneActivitiesImpl returns:

Step 8 ? Controller returns JSON

The controller waits for the CompletableFuture result, then returns status COMPLETED, workflowId, droneId, message, and completedAt.

Step 9 ? React renders response

App.tsx stores the response. The dashboard shows COMPLETED, the message, and the workflow ID. The button is disabled while the request is in progress and displays CONTACTING TEMPORAL....

8. Meaning of the workflow ID

A workflow ID identifies one Temporal execution. It is useful for searching Temporal UI, checking status through the backend, reading workflow history, and diagnosing failures.

To inspect a workflow, open:

Then search for the workflow ID shown by the frontend or returned by PowerShell.

9. Direct API test and what it proves

If status is COMPLETED, this proves the backend is listening, CORS is not relevant to PowerShell, Temporal is reachable, the worker is polling, the workflow was executed, and the activity returned a result.

10. Complete run procedure from scratch

Window 1 ? Docker and Temporal

Window 2 ? Backend

Window 3 ? Frontend

Browser

11. How to verify each layer

Docker and Temporal

Backend

Frontend

Workflow

12. Handling the project during development

Backend change: stop with Ctrl+C, edit Java, run mvn test, then restart mvn spring-boot:run.

Frontend change: Vite usually reloads automatically. If it becomes blank, press Ctrl+F5. If needed, stop with Ctrl+C and restart npm run dev.

Compose change: recreate the Temporal container:

13. Common errors and meanings

Port 8080 already in use

Another Java backend is already running. Test it before starting another one:

If the API succeeds, keep the existing backend. Otherwise inspect the process:

Backend unavailable in frontend

The browser cannot reach port 8080, the backend stopped, or CORS/configuration is wrong. Run the direct API test.

Docker named-pipe error

Docker Desktop is installed but its Linux engine is not ready. Run:

Temporal unhealthy

Frontend blank

Use the exact Vite URL, press Ctrl+F5, confirm npm run dev is still running, and ensure public\cesium contains Assets, ThirdParty, Widgets, and Workers.

14. Scope: what is and is not implemented

Implemented: local Temporal, Docker Compose, Spring Boot REST API, real workflow, Takeoff activity, worker, workflow status endpoint, React dashboard, Cesium globe, three static drones, selection, loading/success/error states, and frontend integration.

Not implemented intentionally: real drone hardware, live GPS, WebSockets, moving markers, navigation, package delivery, return-to-base, authentication, database, cloud deployment, emergency abort, and multi-step missions. These are future features beyond Week 1.

15. Shutdown procedure

In the backend and frontend windows, press Ctrl+C. Then stop Temporal:

Docker Desktop may remain open. To start again, repeat the three-window procedure.

16. Final success checklist

? docker info shows a Server section

? docker compose ps shows Temporal Up and healthy

? backend logs show Started AeroSagaApplication

? backend logs show Workflow Poller and Activity Poller

? frontend prints a Vite Local URL

? dashboard displays the Cesium globe and three drones

? Takeoff returns COMPLETED

? workflow ID appears in the response

? workflow is visible in Temporal UI

A successful run proves: React ? Spring Boot ? Temporal ? worker ? workflow ? activity ? response.

AeroSaga Week 1 ? local simulated drone mission control